



**Universidad
Tecnológica
del Perú**

Universidad Tecnológica del Perú

Análisis y Diseño de Algoritmos

Documentación: Práctica Final

Huatay Salcedo, Luis Elías, U71809640
Ingeniería de Software V

21 de julio de 2026

Docente: Manuel Alejandro, Ortiz Piñella

1. Menú principal

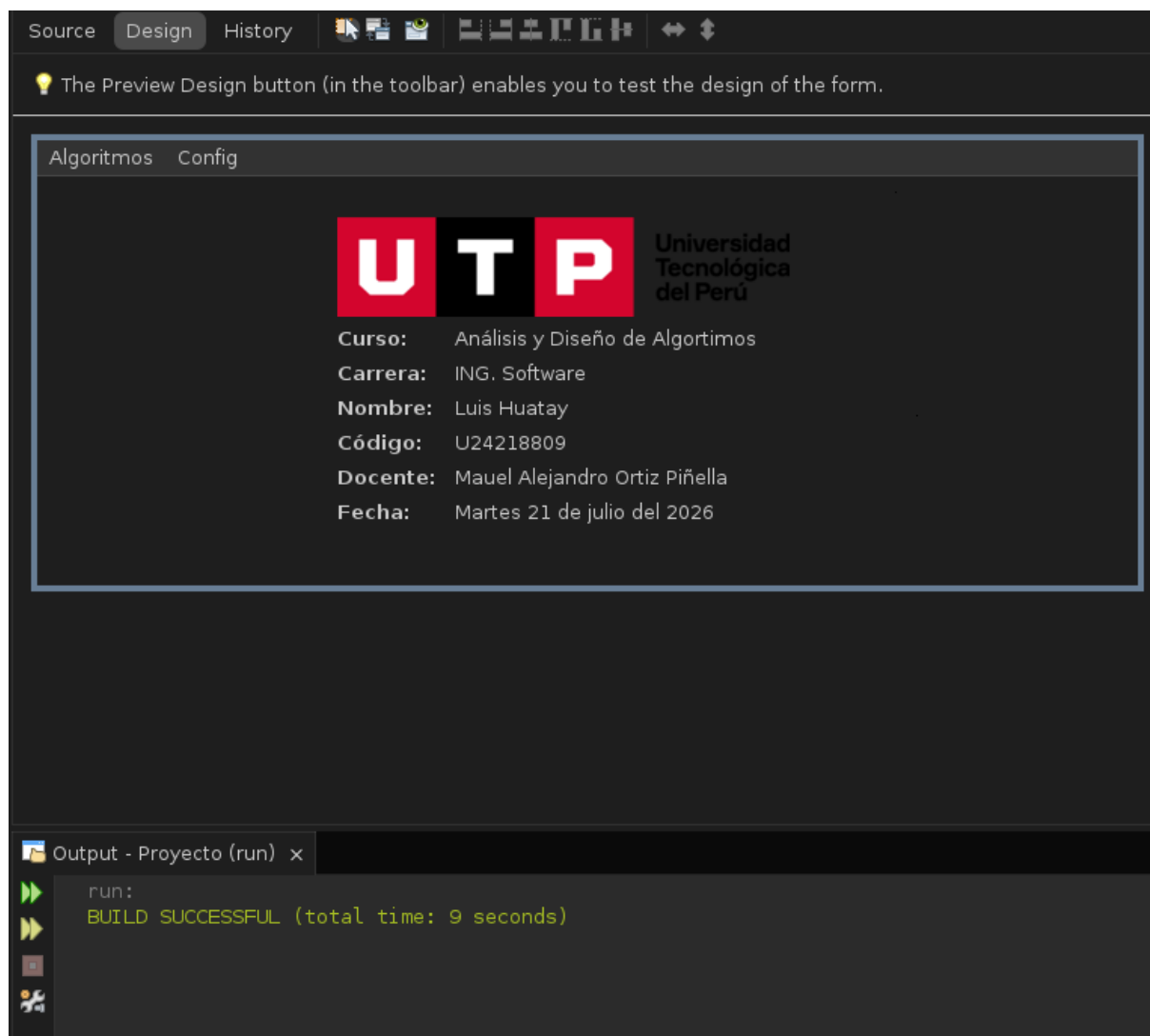


Figura 1: Construcción del Menú con JFrame Form.



Figura 2: Interfaz Menú.

2. Interfaz de Agenad de Mascotas

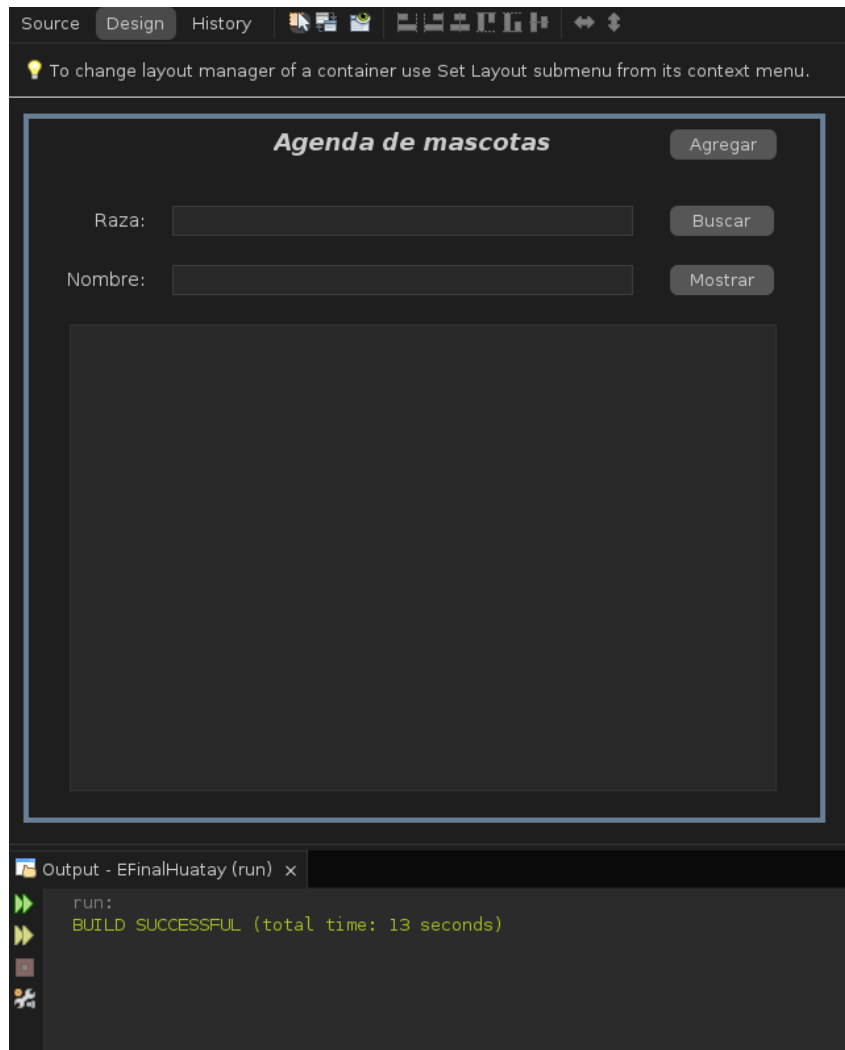


Figura 3: Diseño de la interfaz de Agenda de Mascotas.

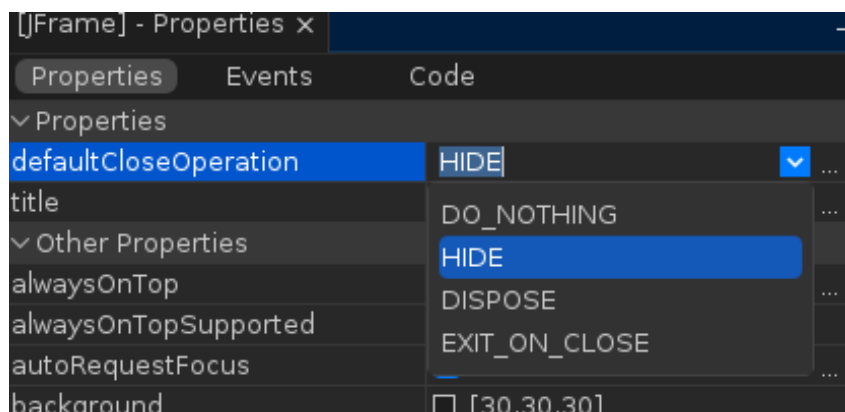


Figura 4: Propiedad que evita que se termine el programa al cerrar la ventana de Agenda de Mascotas.

```
private void btnAgregarActionPerformed(java.awt.event.ActionEvent evt) {  
    String raza = txtRaza.getText();  
    String nombre = txtNombre.getText();  
    if (!raza.isEmpty() && !nombre.isEmpty()) {  
        agenda.put(raza, nombre);  
        txtArea.append("Mascota agregada");  
        txtRaza.setText("");  
        txtNombre.setText("");  
    }  
}
```

Figura 5: Método para agregar los datos de la mascota a la Agenda.

```
private void btnBuscarActionPerformed(java.awt.event.ActionEvent evt) {  
    String raza = txtRaza.getText();  
    if (agenda.containsKey(raza)) {  
        txtArea.setText("El dueño de " + raza + " es: " + agenda.get(raza));  
    } else {  
        txtArea.setText("raza no encontrada.");  
    }  
}  
  
private void btnMostrarActionPerformed(java.awt.event.ActionEvent evt) {  
    StringBuilder sb = new StringBuilder("Lista de razas y nombres:\n");  
    for (String raza : agenda.keySet()) {  
        sb.append(raza).append(" -> ").append(agenda.get(raza)).append("\n");  
    }  
    txtArea.setText(sb.toString());  
}
```

Figura 6: Método para buscar y mostrar los datos de la mascota en la Agenda.

Agenda de mascotas

Raza:

Nombre:

Figura 7: Interfaz de la aplicación de Agenda de Mascotas, agregando 10 registros.

Agenda de mascotas

Raza:

Nombre:

Agregar

Buscar

Mostrar

Lista de razas y nombres:

- labrador -> bobby
- maine -> rosa
- persa -> kity
- siames -> lucia
- bulldog -> king
- golden -> popi
- bengali -> nana
- peruano -> minina
- pastor -> tobi
- chusco -> lucas

Figura 8: Muestra de los registros de la Agenda de Mascotas.

3. Interfaz de ordenamiento alfabético

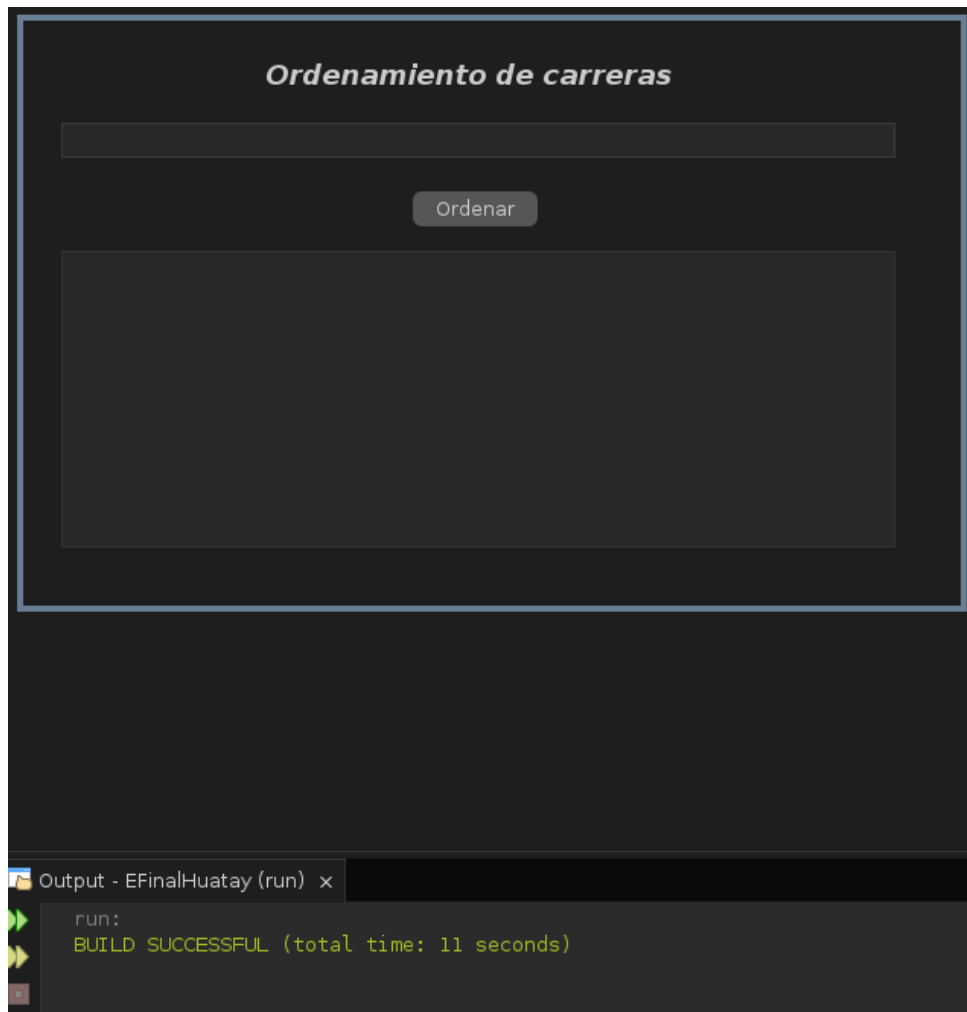


Figura 9: Diseño de la interfaz de ordenamiento alfabético.

```
private void btnOrdenarActionPerformed(java.awt.event.ActionEvent evt) {  
    String texto = txtCarrera.getText().trim();  
  
    if (texto.isEmpty()) {  
        txtArea.setText("Lista original: []\nLista ordenada: []");  
        return;  
    }  
  
    String[] partes = texto.split(",\\s");  
    List<String> listaOriginal = new ArrayList<>();  
    for (String parte : partes) {  
        listaOriginal.add(parte.trim());  
    }  
  
    List<String> listaOrdenada = new ArrayList<>(listaOriginal);  
    Collections.sort(listaOrdenada);  
  
    txtArea.setText("Lista original: " + listaOriginal + "\nLista ordenada: " + listaOrdenada);  
}
```

Figura 10: Método para ordenar los datos alfabéticamente.

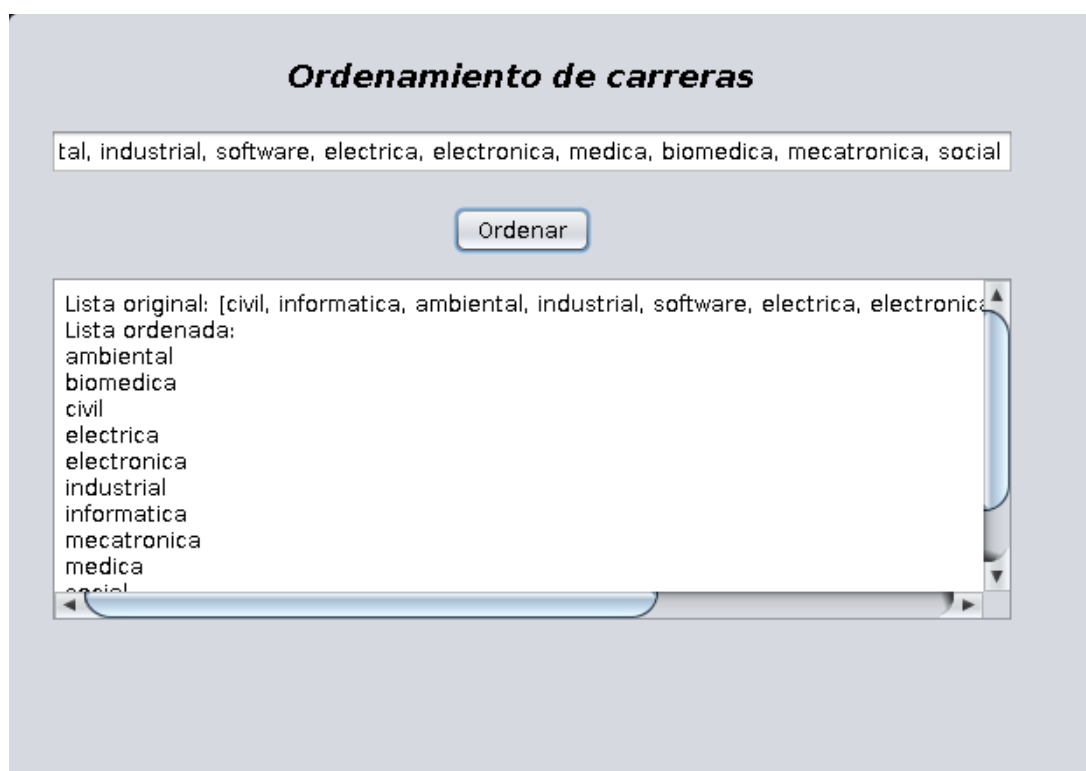


Figura 11: Muestra del funcionamiento de la interfaz de ordenamiento alfabético con los datos del examen.

4. Interfaz de Merge-Sort ascendente y descendente

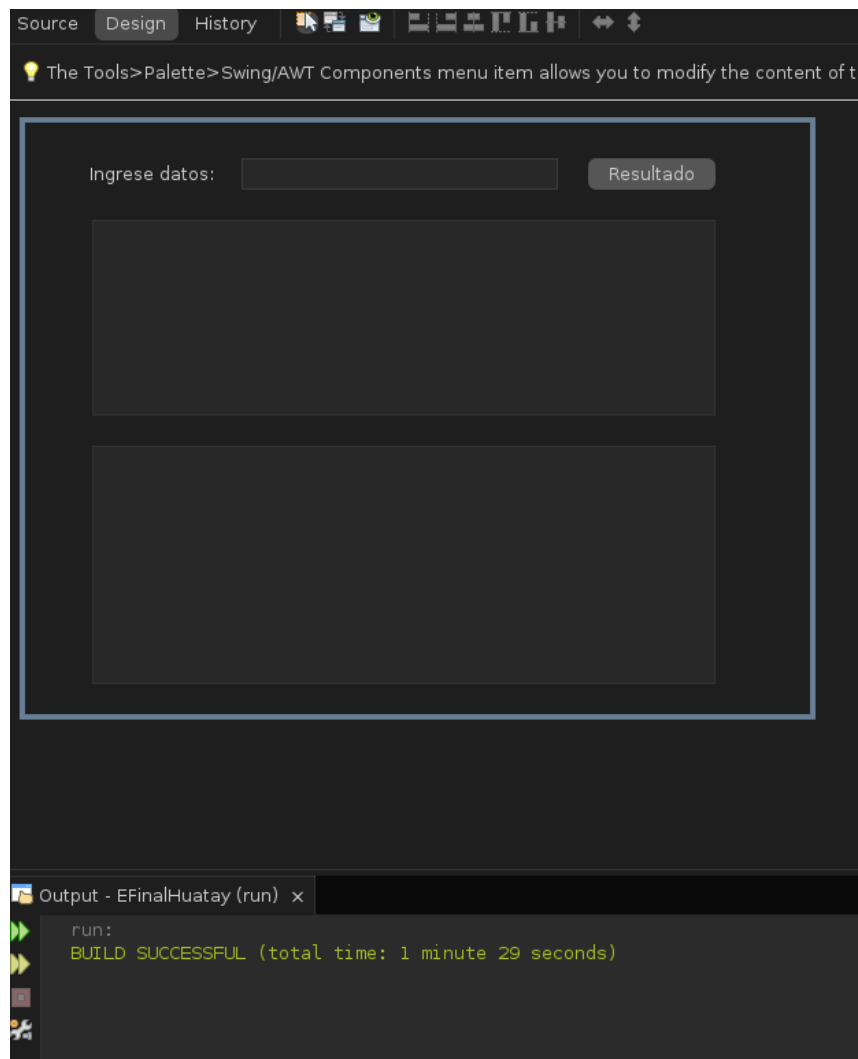


Figura 12: Diseño de la interfaz de Merge-Sort.

```
public void mergeSort(int[] arr, int left, int right) {  
    if (left < right) {  
        int mid = (left + right) / 2;  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
        merge(arr, left, mid, right);  
    }  
}
```

Figura 13: Método para ordenar los datos del examen con Merge-Sort de manera recursiva.

```
public void merge(int[] arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int[] leftArr = new int[n1];
    int[] rightArr = new int[n2];

    System.arraycopy(arr, left, leftArr, 0, n1);
    System.arraycopy(arr, mid + 1, rightArr, 0, n2);

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) {
            arr[k] = leftArr[i];
            i++;
        } else {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = leftArr[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = rightArr[j];
        j++;
        k++;
    }
}
```

Figura 14: Método para unir los datos ordenados.

```
private int[] reverseArray(int[] array) {
    int[] reversed = new int[array.length];
    for (int i = 0; i < array.length; i++) {
        reversed[i] = array[array.length - 1 - i];
    }
    return reversed;
}
```

Figura 15: Método para unir los datos ordenados en orden descendente.

```
private void btnResultadoActionPerformed(java.awt.event.ActionEvent evt) {  
    String input = txtDatos.getText();  
    try {  
        String[] inputArray = input.split(",");  
        numbers = new int[inputArray.length];  
        for (int i = 0; i < inputArray.length; i++) {  
            numbers[i] = Integer.parseInt(inputArray[i].trim());  
        }  
  
        txtArea1.setText("Arreglo original: " + java.util.Arrays.toString(numbers) + "\n");  
        mergeSort(numbers, 0, numbers.length - 1);  
        txtArea1.append("Arreglo ordenado ascendente: " + Arrays.toString(numbers));  
        txtArea2.append("\nArreglo ordenado descendente: " + Arrays.toString(reverseArray(numbers)));  
    } catch (NumberFormatException ex) {  
        JOptionPane.showMessageDialog(null, "Ingresa bien los datos oyeee");  
    }  
}  
} // GEN-LAST:event_btnResultadoActionPerformed
```

Figura 16: Comportamiento del botón de acción en la interfaz.

Ingrese datos: 3, 32, 45, 17, 59, 16, 51, 36, 48, 22 Resultado

Arreglo original: [58, 32, 45, 17, 59, 16, 51, 36, 48, 22]
Arreglo ordenado ascendente:
[16, 17, 22, 32, 36, 45, 48, 51, 58, 59]

Arreglo ordenado descendente:
[59, 58, 51, 48, 45, 36, 32, 22, 17, 16]

Figura 17: Muestra del funcionamiento de la interfaz de Merge-Sort con los datos del examen.